

FDN Class

1.0

Generated by Doxygen 1.17.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 FDNreverb Class Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Function Documentation	6
3.1.2.1 procBlock()	6
3.1.2.2 setabscoef()	6
3.1.2.3 setreflection()	6
3.1.2.4 setroomarea()	7
3.1.2.5 setsamplingrate()	7
3.1.2.6 step()	7
4 File Documentation	9
4.1 FDNreverb.h	9

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

FDNreverb	
Creates a reverb effect using a feedback delay network	5

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

C:/JUCE Projects/FDN Class/ FDNreverb.h	9
--	---

Chapter 3

Class Documentation

3.1 FDNreverb Class Reference

Creates a reverb effect using a feedback delay network.

```
#include <FDNreverb.h>
```

Public Member Functions

- void **setroomarea** (double roomarea)
Set the room area parameter. The room area is used to determine the maximum path length for the delay lines, and thus the sample delay and damping for each delay line. The path lengths are determined based on the golden rectangle, with each subsequent path length being a factor of alpha shorter than the previous one. The alpha parameter is calculated based on the number of delay lines (ORDER) to ensure a good distribution of path lengths.
- void **setabscoef** (double abscoef)
Set absorption coefficient The absorption coefficient is used to determine the damping for each delay line, with higher absorption coefficients resulting in more damping.
- void **setreflection** (double reflection)
Set reflection parameter.
- void **setsamplingrate** (double samplingrate)
Set the sampling rate.
- void **reset** ()
Clear all delay lines.
- double **step** (double sample)
Step FDN reverb through one sample period This method processes a single input sample through the FDN reverb and returns the output sample.
- void **procBlock** (float *samples, unsigned numSamples, double pctReverb=100.0, double dbGain=0.0)
Process a block of audio samples through the FDN reverb. This method processes an array of input samples through the FDN reverb. Each output sample is a weighted mix of the input sample and the reverb output, with the mix determined by the pctReverb parameter. The dbGain parameter allows for adjusting the overall gain of the output signal.

3.1.1 Detailed Description

Creates a reverb effect using a feedback delay network.

3.1.2 Member Function Documentation

3.1.2.1 procBlock()

```
void FDNreverb::procBlock (
    float * samples,
    unsigned numSamples,
    double pctReverb = 100.0,
    double dbGain = 0.0)
```

Process a block of audio samples through the FDN reverb. This method processes an array of input samples through the FDN reverb. Each output sample is a weighted mix of the input sample and the reverb output, with the mix determined by the *pctReverb* parameter. The *dbGain* parameter allows for adjusting the overall gain of the output signal.

Parameters

<i>samples</i>	pointer to the array of input/output samples
<i>numSamples</i>	number of samples in the array
<i>pctReverb</i>	percent reverb (0-100)
<i>dbGain</i>	gain in dB

3.1.2.2 setabscoef()

```
void FDNreverb::setabscoef (
    double abscoef)
```

Set absorption coefficient The absorption coefficient is used to determine the damping for each delay line, with higher absorption coefficients resulting in more damping.

Parameters

<i>abscoef</i>	HF absorption coefficient in dB/m
----------------	-----------------------------------

3.1.2.3 setreflection()

```
void FDNreverb::setreflection (
    double reflection) [inline]
```

Set reflection parameter.

Parameters

<i>reflect</i>	reflection parameter (no units)
----------------	---------------------------------

3.1.2.4 setroomarea()

```
void FDNreverb::setroomarea (
    double roomarea)
```

Set the room area parameter. The room area is used to determine the maximum path length for the delay lines, and thus the sample delay and damping for each delay line. The path lengths are determined based on the golden rectangle, with each subsequent path length being a factor of alpha shorter than the previous one. The alpha parameter is calculated based on the number of delay lines (ORDER) to ensure a good distribution of path lengths.

Parameters

<i>roomarea</i>	room area in m ²
-----------------	-----------------------------

3.1.2.5 setsamplingrate()

```
void FDNreverb::setsamplingrate (
    double samplingrate) [inline]
```

Set the sampling rate.

Parameters

<i>samplingrate</i>	Sampling rate in Hz
---------------------	---------------------

3.1.2.6 step()

```
double FDNreverb::step (
    double sample)
```

Step FDN reverb through one sample period This method processes a single input sample through the FDN reverb and returns the output sample.

Parameters

<i>sample</i>	input sample
---------------	--------------

Returns

output sample

The documentation for this class was generated from the following files:

- C:/JUCE Projects/FDN Class/FDNreverb.h
- C:/JUCE Projects/FDN Class/FDNreverb.cpp

Chapter 4

File Documentation

4.1 FDNreverb.h

```
00001 /*
00002  =====
00003
00004     FDNreverb.h
00005     Created: 24 Nov 2019 12:35:34pm
00006     Author:  profw
00007
00008  =====
00009 */
00010
00011 #pragma once
00012
00013 constexpr auto SPEEDOFSOUND = 343.0; // speed of sound in m/s
00014 constexpr auto ORDER = 6;
00015 constexpr double PHI = 1.618; // golden ratio
00016 constexpr auto MAX_SAMPLE_DELAY = 32000; // maximum delay in samples (for delay line buffer size)
00017
00021 class FDNreverb
00022 {
00023 public:
00024     FDNreverb();
00025     ~FDNreverb() {}
00026
00037     void setroomarea(double roomarea);
00038
00045     void setabscoef(double abscoef);
00046
00051     void setreflection(double reflection) { m_reflection = reflection; }
00052
00057     void setsamplingrate(double samplingrate) { m_samplingRate = samplingrate; }
00058
00062     void reset();
00063
00070     double step(double sample);
00071
00082     void procBlock(float* samples, unsigned numSamples, double pctReverb = 100.0, double dbGain =
00083 0.0);
00084 private:
00085     float m_delayLine[ORDER][MAX_SAMPLE_DELAY];
00086     unsigned m_sampleDelay[ORDER];
00087     double m_damping[ORDER];
00088     double x[ORDER];
00089     double m_alpha; // used to determine path lengths
00090     double m_samplingRate;
00091     double m_absCoef; // HF absorption coefficient in dB/m
00092     double m_reflection; // reflection parameter (no units)
00093     unsigned m_oldest; // index of oldest sample in delay line (for circular buffer)
00094 };
```

